Practice midterm 2 (with solutions)

TC1037 Algorithmic Thinking · Group 510 · Tecnológico de Monterrey, Campus Querétaro

Practice — download, solve on paper in 80 minutes, then check the solutions

Name: _____ **Matrícula:** _____

80 minutes · 100 points · individual · on paper. No computer, phone, notes or calculator. Write in pen or dark pencil; code must be legible and indented. **Part I** (30 pts): write *exactly* what Python prints — every line, in order. **Part II** (70 pts): write the program. Suggested time: 15 minutes for Part I, 65 for Part II. **This is a practice paper: same format, same difficulty, different problems.**

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	5	5	5	5	5	5	23	23	3	20	1	100
Score:												

Part I — Read and predict

(30 points)

1. **Arithmetic operators.** What does Python print? [5]

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

Output:

2. **Boolean operators.** What does Python print? [5]

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

Output:

3. **Conditional structure.** What does Python print? [5]

```
x = 9
y = 9
if x > y:
    print("A")
elif x == y and x % 3 == 0:
    print("B", x * 2)
elif x == y:
    print("C")
else:
    print("D")
print("end")
```

Output:

4. **Functions.** What does Python print? [5]

```python
def k(n):
    n = n - 1
    return n * 2

def main():
    n = 5
    m = k(n)
    m = k(m)
    print(n, m)

main()
```

Output:

5. **The while loop.** What does Python print? [5]

```python
i = 0
while i < 10:
    i = i + 3
    if i % 2 == 0:
        print(i)
print("last", i)
```

Output:

6. **The for loop.** What does Python print? [5]

```python
total = 0
for i in range(5, 0, -2):
    total = total + i
    print(total)
```

Output:

Part II — Write the program **(70 points)**

Write **one** Python program with the following parts. All `input()` and all validation happen in `main()`; the two functions **return** their result and never print or read. **Function 7 must use a `while` loop and function 8 a `for` loop** — each is worth 23 points. Write each function's purpose in a one-line comment above it, and under each loop a one-line comment saying why it stops (`while`) or how many times it runs (`for`).

7. **`while loop`** The function `first_power_above(base, limit)` receives two integers *base* ≥ 2 and *limit* ≥ 1 and **returns** the smallest exponent e such that $base^e > limit$. Use a `while` loop; do not use `math.log`. [23]

   Examples: `first_power_above(2, 10)` → `4` `first_power_above(2, 16)` → `5` `first_power_above(10, 999)` → `3` `first_power_above(3, 1)` → `1`

8. **`for loop`** The function `sum_squares(n)` receives an integer $n \geq 1$ and **returns** $1^2 + 2^2 + \cdots + n^2$. Use a `for` loop. [23]

   Examples: `sum_squares(1)` → `1` `sum_squares(3)` → `14` `sum_squares(4)` → `30` `sum_squares(10)` → `385`

9. The function `menu()` that displays: `1. first_power_above  2. sum_squares  3. Exit` [3]

10. The function `main()` that, inside a `while` loop: shows the menu, reads the option, and [20]
    - option 1: reads base (2 or more) and limit (1 or more); if any value is out of range displays a clear message, otherwise calls `first_power_above` and displays the result with a label;
    - option 2: reads n (must be 1 or more); same rule, calling `sum_squares`;
    - option 3: displays `Goodbye.` and the program ends;
    - any other option: displays `Invalid option.` and the program ends.

    Use `if`/`elif`/`else` to choose the option.

11. The last line of the script calls `main()`. [1]

Technical requirements — 2 points off for each one missing: meaningful variable names in Python style · a clear prompt for every input · readable, labelled output.

Part II — your program (continue on the back if needed)

How Part II is graded (the academia's rubric)

Each part of the program (the two functions, `menu`, `main`, the call) is scored on this scale, as a percentage of its points:

Excelente 100%	**Muy bien 85%**	**Regular 70%**	**Deficiente 50%**	**Muy deficiente 20%**	**Sin evidencia 0%**
No syntax errors; works correctly, including the boundary cases in the examples.	Correct approach; minimal logic or syntax slips.	At most two syntax errors and/or small logic errors; the approach is closely related to what was asked.	Approach loosely related, or related but with more than two syntax errors.	Code unrelated to the problem.	No code.

Part I: 5 points per question, all or nothing — every line of the output, in order. Technical requirements: -2 each, from Part II.